

Программирование на Python, БИ

НИУ ВШЭ, 2025–26 учебный год

✓ Семинар 9. EDA

В качестве эпиграфа к нашему сегодняшнему семинару, представляем вашему вниманию любопытный список из некоторых этапов:

- Извлечение данных
- Подготовка данных — очистка данных
- Подготовка данных — преобразование данных
- Исследование и визуализация данных
- Инжиниринг признаков

Вот оказывается, все эти пункты для DS'ника — это зачастую на первый взгляд скрытые, но одновременно с этим решительно необходимые этапы, по факту и позволяющие осуществить всю ту магию, которой так славится машинное обучение! В принципе, всё самое важное часто остаётся в тени, и EDA в этом плане — не исключение. Но это, конечно, абсолютно нормально! :)

Сегодня мы с вами попробуем лишь слегка прикоснуться к EDA и бегло посмотреть на основные этапы, которые реализуются в рамках этого процесса, а также повторить различные инструменты визуализаций и работы с данными, которые мы разбирали на предыдущих занятиях.

И да — то, что весь этот огромный ноутбук с миллионом разделов — это "лишь слегка прикоснуться", — это вы не ослышались... Думаю, вы действительно сможете почувствовать на практике, насколько EDA объёмен, важен, да и вообще масштаб всей нашей работы, — после того, как мы закончим!

Ну что же — кажется, пора приступать! :))

✓ Постановка задачи

Постановка задачи и оценка имеющихся данных — первый и важнейший шаг на пути к решению. Сделать это нужно ещё до того, как будет написана первая строка кода.

Для нашего сегодняшнего семинара и, как следствие, для нашей задачи, данные — это открытые сведения об [энергопотреблении зданий в Нью-Йорке](#).

Наша цель — предсказать рейтинг энергопотребления здания (Energy Star Score) и понять, какие признаки оказывают на него наибольшее влияние.

Поехали!

✓ Чистка данных

Вопреки тому, какое впечатление может сложиться после посещения различных курсов и чтения статей по машинному обучению, данные не всегда представляют собой идеально организованный набор наблюдений без каких-либо пропусков или аномалий (например, можно взглянуть на известные наборы данных `mtcars` и `iris`). Обычно данные содержат в себе кучу мусора, который необходимо почистить, да и вообще сами данные порой лучше воспринимать критически, для того чтобы затем привести их в приемлемый формат.

Чистка данных — необходимый и важнейший этап решения почти любой реальной задачи.

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from IPython.core.pylabtools import figsize
```

```
# Это не обязательно, но позволяет красиво отображать информацию в блокноте

%matplotlib inline

plt.rcParams['font.size'] = 24

pd.options.mode.chained_assignment = None
pd.set_option('display.max_columns', 60)

sns.set(font_scale = 2)
```

✓ Считывание данных

Считаем данные и посмотрим на первые строки

```
data = pd.read_csv('https://raw.githubusercontent.com/ddvika/DS_2021/main/lecture_2/Energy_consumption_NY.csv')
data.head()
```

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Address 2	Postal Code	Street Number	Street Name	Borough	DOF Gross Floor Area	Primary Property Type – Self Selected	List of All Property Use Types at Property
0	1	13286	201/205	13286	201/205	1013160001	1013160001	1037549	201/205 East 42nd st.	Not Available	10017	675	3 AVENUE	Manhattan	289356.0	Office	Office
1	2	28400	NYP Columbia (West Campus)	28400	NYP Columbia (West Campus)	1021380040	1-02138-0040	1084198; 1084387; 1084385; 1084386; 1084388; 10...	622 168th Street	Not Available	10032	180	WASHINGTON AVENUE	Manhattan	3693539.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
2	3	4778226	MSCHoNY North	28400	NYP Columbia (West Campus)	1021380030	1-02138-0030	1063380	3975 Broadway	Not Available	10032	3975	BROADWAY	Manhattan	152765.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
3	4	4778267	Herbert Irving Pavilion & Millstein Hospital	28400	NYP Columbia (West Campus)	1021390001	1-02139-0001	1087281; 1076746	161 Fort Washington Ave	177 Fort Washington Ave	10032	161	WASHINGTON AVENUE	Manhattan	891040.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
4	5	4778288	Neuro Institute	28400	NYP Columbia (West Campus)	1021390085	1-02139-0085	1063403	710 West 168th Street	Not Available	10032	193	WASHINGTON AVENUE	Manhattan	211400.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)

```
print("Размеры датасета: ", data.shape)
print("Колонки датасета: ", data.columns)

Размеры датасета: (11746, 60)
Колонки датасета: Index(['Order', 'Property Id', 'Property Name', 'Parent Property Id',
      'Parent Property Name', 'BBL – 10 digits',
      'NYC Borough, Block and Lot (BBL) self-reported',
      'NYC Building Identification Number (BIN)', 'Address 1 (self-reported)',
      'Address 2', 'Postal Code', 'Street Number', 'Street Name', 'Borough',
      'DOF Gross Floor Area', 'Primary Property Type – Self Selected',
      'List of All Property Use Types at Property',
      'Largest Property Use Type',
      'Largest Property Use Type – Gross Floor Area (ft²)',
      '2nd Largest Property Use Type',
      '2nd Largest Property Use – Gross Floor Area (ft²)',
      '3rd Largest Property Use Type',
      '3rd Largest Property Use Type – Gross Floor Area (ft²)', 'Year Built',
      'Number of Buildings – Self-reported', 'Occupancy',
      'Metered Areas (Energy)', 'Metered Areas (Water)', 'ENERGY STAR Score',
      'Site EUI (kBtu/ft²)', 'Weather Normalized Site EUI (kBtu/ft²)',
      'Weather Normalized Site Electricity Intensity (kWh/ft²)',
      'Weather Normalized Site Natural Gas Intensity (therms/ft²)',
      'Weather Normalized Source EUI (kBtu/ft²)', 'Fuel Oil #1 Use (kBtu)',
```

```
'Fuel Oil #2 Use (kBtu)', 'Fuel Oil #4 Use (kBtu)',
'Fuel Oil #5 & 6 Use (kBtu)', 'Diesel #2 Use (kBtu)',
'District Steam Use (kBtu)', 'Natural Gas Use (kBtu)',
'Weather Normalized Site Natural Gas Use (therms)',
'Electricity Use - Grid Purchase (kBtu)',
'Weather Normalized Site Electricity (kWh)',
'Total GHG Emissions (Metric Tons CO2e)',
'Direct GHG Emissions (Metric Tons CO2e)',
'Indirect GHG Emissions (Metric Tons CO2e)',
'Property GFA - Self-Reported (ft²)',
'Water Use (All Water Sources) (kgal)',
'Water Intensity (All Water Sources) (gal/ft²)',
'Source EUI (kBtu/ft²)', 'Release Date', 'Water Required?',
'DOF Benchmarking Submission Status', 'Latitude', 'Longitude',
'Community Board', 'Council District', 'Census Tract', 'NTA'],
dtype='object')
```

Воспользуемся методом info для того, чтобы получить представление о столбцах датафрейма

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11746 entries, 0 to 11745
Data columns (total 60 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Order                                     11746 non-null  int64
1   Property Id                             11746 non-null  int64
2   Property Name                           11746 non-null  object
3   Parent Property Id                      11746 non-null  object
4   Parent Property Name                    11746 non-null  object
5   BBL - 10 digits                         11735 non-null  object
6   NYC Borough, Block and Lot (BBL) self-reported 11746 non-null  object
7   NYC Building Identification Number (BIN) 11746 non-null  object
8   Address 1 (self-reported)               11746 non-null  object
9   Address 2                              11746 non-null  object
10  Postal Code                             11746 non-null  object
11  Street Number                           11622 non-null  object
12  Street Name                             11624 non-null  object
13  Borough                                 11628 non-null  object
14  DOF Gross Floor Area                    11628 non-null  float64
15  Primary Property Type - Self Selected    11746 non-null  object
16  List of All Property Use Types at Property 11746 non-null  object
17  Largest Property Use Type                11746 non-null  object
18  Largest Property Use Type - Gross Floor Area (ft²) 11746 non-null  object
19  2nd Largest Property Use Type            11746 non-null  object
20  2nd Largest Property Use - Gross Floor Area (ft²) 11746 non-null  object
21  3rd Largest Property Use Type            11746 non-null  object
22  3rd Largest Property Use Type - Gross Floor Area (ft²) 11746 non-null  object
23  Year Built                              11746 non-null  int64
24  Number of Buildings - Self-reported      11746 non-null  int64
25  Occupancy                              11746 non-null  int64
26  Metered Areas (Energy)                  11746 non-null  object
27  Metered Areas (Water)                   11746 non-null  object
28  ENERGY STAR Score                      11746 non-null  object
29  Site EUI (kBtu/ft²)                     11746 non-null  object
30  Weather Normalized Site EUI (kBtu/ft²)  11746 non-null  object
31  Weather Normalized Site Electricity Intensity (kWh/ft²) 11746 non-null  object
```

32	Weather Normalized Site Natural Gas Intensity (therms/ft²)	11746	non-null	object
33	Weather Normalized Source EUI (kBtu/ft²)	11746	non-null	object
34	Fuel Oil #1 Use (kBtu)	11746	non-null	object
35	Fuel Oil #2 Use (kBtu)	11746	non-null	object
36	Fuel Oil #4 Use (kBtu)	11746	non-null	object
37	Fuel Oil #5 & 6 Use (kBtu)	11746	non-null	object
38	Diesel #2 Use (kBtu)	11746	non-null	object
39	District Steam Use (kBtu)	11746	non-null	object
40	Natural Gas Use (kBtu)	11746	non-null	object
41	Weather Normalized Site Natural Gas Use (therms)	11746	non-null	object
42	Electricity Use – Grid Purchase (kBtu)	11746	non-null	object
43	Weather Normalized Site Electricity (kWh)	11746	non-null	object
44	Total GHG Emissions (Metric Tons CO2e)	11746	non-null	object
45	Direct GHG Emissions (Metric Tons CO2e)	11746	non-null	object
46	Indirect GHG Emissions (Metric Tons CO2e)	11746	non-null	object
47	Property GFA – Self-Reported (ft²)	11746	non-null	int64
48	Water Use (All Water Sources) (kgal)	11746	non-null	object
49	Water Intensity (All Water Sources) (gal/ft²)	11746	non-null	object
50	Source EUI (kBtu/ft²)	11746	non-null	object
51	Release Date	11746	non-null	object
52	Water Required?	11628	non-null	object

После непродолжительного обзора данных, в глаза сразу бросается пара проблем. Какие?

- 1. Мы уже знаем, что хотим предсказать ENERGY STAR Score, но хорошо бы понять, что из себя представляют остальные признаки. Это не всегда проблема, иногда удастся решить задачу машинного обучения, не имея почти никакого представления о том, что признаки из себя представляют. Однако в нашем случае, для нас важна интерпретируемость, поэтому важно понимать, что несут в себе основные характеристики данных.
- 2. Не все признаки для нас одинаково важны, однако с нашим целевым признаком уж точно нужно разобраться! Вот какое описание для целевой переменной нам представлено на сайте-источнике: «Оценка в баллах от 1 до 100, основанная на предоставленных сведениях о потреблении электроэнергии. Рейтинг энергопотребления это относительная величина, необходимая для сравнения эффективности использования энергии различными зданиями.»
- 3. Пропущенные данные, вставленные в набор, выглядят как строка с записью “Not Available”. Это означает, что Python, даже если эта колонка содержит в себе преимущественно числовые признаки, будет интерпретировать её как тип данных object, потому что Pandas интерпретирует любой признак, содержащий строковые значения, как строку.

✦ Корректировка типов

Еще раз воспользуемся методом `data.info()`

```
data.info()
```

```

13 borough
14 DOF Gross Floor Area
15 Primary Property Type – Self Selected
16 List of All Property Use Types at Property
17 Largest Property Use Type
18 Largest Property Use Type – Gross Floor Area (ft²)
19 2nd Largest Property Use Type
20 2nd Largest Property Use – Gross Floor Area (ft²)
21 3rd Largest Property Use Type
22 3rd Largest Property Use Type – Gross Floor Area (ft²)
23 Year Built
24 Number of Buildings – Self-reported
25 Occupancy
26 Metered Areas (Energy)
27 Metered Areas (Water)
28 ENERGY STAR Score
29 Site EUI (kBtu/ft²)
30 Weather Normalized Site EUI (kBtu/ft²)
31 Weather Normalized Site Electricity Intensity (kWh/ft²)
32 Weather Normalized Site Natural Gas Intensity (therms/ft²)
33 Weather Normalized Source EUI (kBtu/ft²)
34 Fuel Oil #1 Use (kBtu)
35 Fuel Oil #2 Use (kBtu)
36 Fuel Oil #4 Use (kBtu)
37 Fuel Oil #5 & 6 Use (kBtu)
38 Diesel #2 Use (kBtu)
39 District Steam Use (kBtu)
40 Natural Gas Use (kBtu)
41 Weather Normalized Site Natural Gas Use (therms)
42 Electricity Use – Grid Purchase (kBtu)
43 Weather Normalized Site Electricity (kWh)
44 Total GHG Emissions (Metric Tons CO2e)
45 Direct GHG Emissions (Metric Tons CO2e)
46 Indirect GHG Emissions (Metric Tons CO2e)
47 Property GFA – Self-Reported (ft²)
48 Water Use (All Water Sources) (kgal)
49 Water Intensity (All Water Sources) (gal/ft²)
50 Source EUI (kBtu/ft²)
51 Release Date
52 Water Required?
53 DOF Benchmarking Submission Status
54 Latitude
55 Longitude
56 Community Board
57 Council District
58 Census Tract
59 NTA
dtypes: float64(6), int64(6), object(48)
memory usage: 5.4+ MB

```

Очевидно, многие признаки, являющиеся изначально числовыми (например, площади), интерпретированы как object.

Анализировать их крайне сложно, так что сначала конвертируем их в числа, а именно в тип float.

Заменим значение “Not Available” в данных на «не число» (np.nan — «not a number»), которое Python все же интерпретирует как число. Это позволит изменить тип соответствующих числовых признаков на float:

```
# Для замены значений в датафрейме воспользуемся методом replace({from_val1: to_val1, ...})

data = data.replace({'Not Available': np.nan})
```

Теперь давайте найдем колонки, которые должны быть числовыми.

Как это сделать? Есть предложения?

data.sample(3)

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Address 2	Postal Code	Street Number	Street Name	Borough	DOF Gross Floor Area	Primary Property Type – Self Selected	List of Property Types
5986	7108	2805189	1460 Grand Concourse - 35383	Not Applicable: Standalone Property	Not Applicable: Standalone Property	2028320035	2-02832-0035	2007992	1460 Grand Concourse	NaN	10457	1460	GRAND CONCOURSE	Bronx	55776.0	Multifamily Housing	Multifa Hou
1907	2250	4405918	Madison Ave	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1008580058	1-00858-0058	1016920	89-95 Madison Ave	NaN	10016	89	MADISON AVENUE	Manhattan	146161.0	Office	Of R S
4267	4859	2815700	145 East 92 Street	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1015210020	1-01521-0020	1048344	145 East 92 Street	NaN	10128	145	EAST 92 STREET	Manhattan	85924.0	Multifamily Housing	Multifa Hou

Все гениальное просто: мы найдем, какие подстроки должны содержаться в названиях таких столбцов

```
part_name_numeric_cols = ['ft²', 'kBtu', 'Metric Tons CO2e', 'kWh', 'therms', 'gal', 'Score']
```

Конвертируем выбранные числовые столбцы в тип float (например воспользовавшись генератором и функцией any)

```
for col in list(data.columns):
    if any([n_col in col for n_col in part_name_numeric_cols]):
        print(col)
        data[col] = data[col].astype(float)
```

Largest Property Use Type – Gross Floor Area (ft²)
2nd Largest Property Use – Gross Floor Area (ft²)
3rd Largest Property Use Type – Gross Floor Area (ft²)
ENERGY STAR Score
Site EUI (kBtu/ft²)
Weather Normalized Site EUI (kBtu/ft²)
Weather Normalized Site Electricity Intensity (kWh/ft²)

Weather Normalized Site Natural Gas Intensity (therms/ft²)
Weather Normalized Source EUI (kBtu/ft²)
Fuel Oil #1 Use (kBtu)
Fuel Oil #2 Use (kBtu)
Fuel Oil #4 Use (kBtu)
Fuel Oil #5 & 6 Use (kBtu)
Diesel #2 Use (kBtu)
District Steam Use (kBtu)
Natural Gas Use (kBtu)
Weather Normalized Site Natural Gas Use (therms)
Electricity Use – Grid Purchase (kBtu)
Weather Normalized Site Electricity (kWh)
Total GHG Emissions (Metric Tons CO2e)
Direct GHG Emissions (Metric Tons CO2e)
Indirect GHG Emissions (Metric Tons CO2e)
Property GFA – Self-Reported (ft²)
Water Use (All Water Sources) (kgal)
Water Intensity (All Water Sources) (gal/ft²)
Source EUI (kBtu/ft²)

Проверим, что всё получилось:

data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11746 entries, 0 to 11745
Data columns (total 60 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Order                                     11746 non-null  int64
1   Property Id                             11746 non-null  int64
2   Property Name                           11746 non-null  object
3   Parent Property Id                     11746 non-null  object
4   Parent Property Name                   11746 non-null  object
5   BBL – 10 digits                        11735 non-null  object
6   NYC Borough, Block and Lot (BBL) self-reported 11735 non-null  object
7   NYC Building Identification Number (BIN) 11584 non-null  object
8   Address 1 (self-reported)              11746 non-null  object
9   Address 2                              207 non-null   object
10  Postal Code                             11746 non-null  object
11  Street Number                           11622 non-null  object
12  Street Name                             11624 non-null  object
13  Borough                                 11628 non-null  object
14  DOF Gross Floor Area                   11628 non-null  float64
15  Primary Property Type – Self Selected  11746 non-null  object
16  List of All Property Use Types at Property 11746 non-null  object
17  Largest Property Use Type               11744 non-null  object
18  Largest Property Use Type – Gross Floor Area (ft²) 11744 non-null  float64
19  2nd Largest Property Use Type           3741 non-null  object
20  2nd Largest Property Use – Gross Floor Area (ft²) 3741 non-null  float64
21  3rd Largest Property Use Type           1484 non-null  object
22  3rd Largest Property Use Type – Gross Floor Area (ft²) 1484 non-null  float64
23  Year Built                             11746 non-null  int64
24  Number of Buildings – Self-reported     11746 non-null  int64
25  Occupancy                             11746 non-null  int64
26  Metered Areas (Energy)                  11689 non-null  object
27  Metered Areas (Water)                   7137 non-null  object
28  ENERGY STAR Score                     9642 non-null  float64
```

29	Site EUI (kBtu/ft²)	11583 non-null	float64
30	Weather Normalized Site EUI (kBtu/ft²)	10281 non-null	float64
31	Weather Normalized Site Electricity Intensity (kWh/ft²)	10959 non-null	float64
32	Weather Normalized Site Natural Gas Intensity (therms/ft²)	9783 non-null	float64
33	Weather Normalized Source EUI (kBtu/ft²)	10281 non-null	float64
34	Fuel Oil #1 Use (kBtu)	9 non-null	float64
35	Fuel Oil #2 Use (kBtu)	2581 non-null	float64
36	Fuel Oil #4 Use (kBtu)	1321 non-null	float64
37	Fuel Oil #5 & 6 Use (kBtu)	594 non-null	float64
38	Diesel #2 Use (kBtu)	16 non-null	float64
39	District Steam Use (kBtu)	936 non-null	float64
40	Natural Gas Use (kBtu)	10304 non-null	float64
41	Weather Normalized Site Natural Gas Use (therms)	9784 non-null	float64
42	Electricity Use – Grid Purchase (kBtu)	11502 non-null	float64
43	Weather Normalized Site Electricity (kWh)	10960 non-null	float64
44	Total GHG Emissions (Metric Tons CO2e)	11672 non-null	float64
45	Direct GHG Emissions (Metric Tons CO2e)	11663 non-null	float64
46	Indirect GHG Emissions (Metric Tons CO2e)	11681 non-null	float64
47	Property GFA – Self-Reported (ft²)	11746 non-null	float64
48	Water Use (All Water Sources) (kgal)	7762 non-null	float64
49	Water Intensity (All Water Sources) (gal/ft²)	7762 non-null	float64
50	Source EUI (kBtu/ft²)	11583 non-null	float64
51	Release Date	11746 non-null	object
52	Water Required?	11628 non-null	object

Гениально, это сработало! Мы молодцы! :)

Посчитаем, сколько после всех манипуляций у нас стало пропущенных значений:

```
data.isna().sum()
```


	0
Order	0
Property Id	0
Property Name	0
Parent Property Id	0
Parent Property Name	0
BBL - 10 digits	11
NYC Borough, Block and Lot (BBL) self-reported	11
NYC Building Identification Number (BIN)	162
Address 1 (self-reported)	0
Address 2	11539
Postal Code	0
Street Number	124
Street Name	122
Borough	118
DOF Gross Floor Area	118
Primary Property Type - Self Selected	0
List of All Property Use Types at Property	0
Largest Property Use Type	2
Largest Property Use Type - Gross Floor Area (ft²)	2
2nd Largest Property Use Type	8005
2nd Largest Property Use - Gross Floor Area (ft²)	8005
3rd Largest Property Use Type	10262
3rd Largest Property Use Type - Gross Floor Area (ft²)	10262
Year Built	0
Number of Buildings - Self-reported	0
Occupancy	0
Metered Areas (Energy)	57
Metered Areas (Water)	4609
ENERGY STAR Score	2104
Site EUI (kBtu/ft²)	163
Weather Normalized Site EUI (kBtu/ft²)	1465

Выглядит, честно говоря, не очень приятно...

Посмотрим на статистику по колонкам датасета при помощи метода `data.describe()`

```
data.describe()
```

	Weather Normalized Site Natural Gas Intensity (therms/ft²)	1963											
	Weather Normalized Source EUI (kBtu/ft²)	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area
	Order	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area	Property DOF Gross Floor Area
	Fuel Oil #1 Use (kBtu)	Fuel Oil #2 Use (kBtu)	Fuel Oil #3 Use (kBtu)	Fuel Oil #4 Use (kBtu)	Fuel Oil #5 Use (kBtu)	Fuel Oil #6 Use (kBtu)	Fuel Oil #7 Use (kBtu)	Fuel Oil #8 Use (kBtu)	Fuel Oil #9 Use (kBtu)	Fuel Oil #10 Use (kBtu)	Fuel Oil #11 Use (kBtu)	Fuel Oil #12 Use (kBtu)	Fuel Oil #13 Use (kBtu)
	11746.000000	1.174600e+04	1.162800e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04
	count	11746.000000	1.174600e+04	1.162800e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04	1.174400e+04
	mean	7185.759578	3.642958e+06	1.732695e+05	1.605524e+05	1.605524e+05	1.605524e+05	1.605524e+05	1.605524e+05	1.605524e+05	1.605524e+05	1.605524e+05	1.605524e+05
	std	4323.859984	1.049070e+06	3.367055e+05	3.095746e+05	3.095746e+05	3.095746e+05	3.095746e+05	3.095746e+05	3.095746e+05	3.095746e+05	3.095746e+05	3.095746e+05
	min	1.000000	7.365000e+03	5.002800e+04	5.400000e+01	5.400000e+01	5.400000e+01	5.400000e+01	5.400000e+01	5.400000e+01	5.400000e+01	5.400000e+01	5.400000e+01
	25%	3428.250000	2.747222e+06	6.524000e+04	6.520100e+04	6.520100e+04	6.520100e+04	6.520100e+04	6.520100e+04	6.520100e+04	6.520100e+04	6.520100e+04	6.520100e+04
	50%	6986.500000	3.236404e+06	9.313850e+04	9.132400e+04	9.132400e+04	9.132400e+04	9.132400e+04	9.132400e+04	9.132400e+04	9.132400e+04	9.132400e+04	9.132400e+04
	75%	11054.500000	4.409092e+06	1.596140e+05	1.532550e+05	1.532550e+05	1.532550e+05	1.532550e+05	1.532550e+05	1.532550e+05	1.532550e+05	1.532550e+05	1.532550e+05
	max	14993.000000	5.991312e+06	1.354011e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07
	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)	Weather Normalized Site Electricity (kWh)
	14993.000000	5.991312e+06	1.354011e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07	1.421712e+07
	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)	Total GHG Emissions (Metric Tons CO2e)
	74	74	74	74	74	74	74	74	74	74	74	74	74
	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)	Direct GHG Emissions (Metric Tons CO2e)
	83	83	83	83	83	83	83	83	83	83	83	83	83
	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)	Indirect GHG Emissions (Metric Tons CO2e)
	65	65	65	65	65	65	65	65	65	65	65	65	65

Почему при использовании `data.describe()` вывелась статистика не по всем столбцам (всего их 60)?

	Water Use (All Water Sources) (kgal)	3984
data.describe().shape		
(8, 37)	Source EUI (kBtu/ft²)	163
	Release Date	0
data.shape		
(11746, 60)	DOF Benchmarking Submission Status	30

	Latitude	2263
	Longitude	2263
	Community Board	2263
	Council District	2263
	Census Tract	2263
	NTA	2263

Детектирование пропусков

Для начала давайте попробуем оценить масштаб проблемы для наших данных

```
# Воспользуемся следующей функцией, которая предоставляет отчет о пропусках в данных

def missing_values_table(df):
```

```
"""
Функция возвращает резюме по пропущенным значениям
"""

# Общее число пропусков
mis_val = df.isnull().sum()

# Процент пропусков
mis_val_percent = 100 * df.isnull().sum() / len(df)

# Создадит таблицу с результатом
mis_val_table = pd.concat([mis_val, mis_val_percent], axis=1)

# Переименнуем колонки
mis_val_table_ren_columns = mis_val_table.rename(
    columns = {0 : 'Missing Values', 1 : '% of Total Values'})

# Отсортируем по проценту пропущенных значений
mis_val_table_ren_columns = mis_val_table_ren_columns[
    mis_val_table_ren_columns.iloc[:,1] != 0].sort_values(
'% of Total Values', ascending=False).round(1)

# Выведем некоторую информацию
print ("Your selected dataframe has " + str(df.shape[1]) + " columns.\n"
      "There are " + str(mis_val_table_ren_columns.shape[0]) +
      " columns that have missing values.")

return mis_val_table_ren_columns
```

```
# Воспользуемся описанной выше функцией на нашем датасете
```

```
missing_values_table(data)
```

Your selected dataframe has 60 columns.
There are 46 columns that have missing values.

	Missing Values	% of Total Values
Fuel Oil #1 Use (kBtu)	11737	99.9
Diesel #2 Use (kBtu)	11730	99.9
Address 2	11539	98.2
Fuel Oil #5 & 6 Use (kBtu)	11152	94.9
District Steam Use (kBtu)	10810	92.0
Fuel Oil #4 Use (kBtu)	10425	88.8
3rd Largest Property Use Type - Gross Floor Area (ft²)	10262	87.4
3rd Largest Property Use Type	10262	87.4
Fuel Oil #2 Use (kBtu)	9165	78.0
2nd Largest Property Use Type	8005	68.2
2nd Largest Property Use - Gross Floor Area (ft²)	8005	68.2
Metered Areas (Water)	4609	39.2
Water Intensity (All Water Sources) (gal/ft²)	3984	33.9
Water Use (All Water Sources) (kgal)	3984	33.9
Latitude	2263	19.3
Longitude	2263	19.3
Community Board	2263	19.3
Council District	2263	19.3
Census Tract	2263	19.3
NTA	2263	19.3
ENERGY STAR Score	2104	17.9
Weather Normalized Site Natural Gas Intensity (therms/ft²)	1963	16.7
Weather Normalized Site Natural Gas Use (therms)	1962	16.7
Weather Normalized Source EUI (kBtu/ft²)	1465	12.5
Weather Normalized Site EUI (kBtu/ft²)	1465	12.5
Natural Gas Use (kBtu)	1442	12.3
Weather Normalized Site Electricity Intensity (kWh/ft²)	787	6.7
Weather Normalized Site Electricity (kWh)	786	6.7
Electricity Use - Grid Purchase (kBtu)	244	2.1
Site EUI (kBtu/ft²)	163	1.4
Source EUI (kBtu/ft²)	163	1.4

При удалении данных следует быть осторожным! Тем не менее, признак нам вряд ли вообще пригодится, если пропусков в нем слишком много. В данном случае удаляем признаки, в которых пропусков больше 50%.

Street Name	122	1.0
# Получим отчет из функции missing_values_table missing_df = missing_values_table(data)		
# Выберем те колонки, для которых '% of Total Values' больше 50 missing_columns = list(missing_df[missing_df['% of Total Values'] > 50].index)		
print('We will remove %d columns.' % len(missing_columns))		
Total GHG Emissions (Metric Tons CO2e)	74	0.6
Your selected dataframe has 60 columns. There are 4 columns that have missing values. We will remove 11 columns.		
Indirect GHG Emissions (Metric Tons CO2e)	65	0.6
Metered Areas (Energy)	57	0.5

# Удалим столбцы с большим числом пропусков		
data = data.drop(columns = list(missing_columns))		
DOE Regions	11	0.1

# Проверим, что датасет (в частности его размер) изменился ожидаемым образом		
data.shape		
(11746, 49)		

# Посмотрим еще раз на пропуски при помощи функции missing_values_table		
missing_values_table(data)		

Your selected dataframe has 49 columns.
There are 35 columns that have missing values.

	Missing Values	% of Total Values
Metered Areas (Water)	4609	39.2
Water Intensity (All Water Sources) (gal/ft²)	3984	33.9
Water Use (All Water Sources) (kgal)	3984	33.9
NTA	2263	19.3
Census Tract	2263	19.3
Council District	2263	19.3
Community Board	2263	19.3
Longitude	2263	19.3
Latitude	2263	19.3
ENERGY STAR Score	2104	17.9
Weather Normalized Site Natural Gas Intensity (therms/ft²)	1963	16.7
Weather Normalized Site Natural Gas Use (therms)	1962	16.7
Weather Normalized Source EUI (kBtu/ft²)	1465	12.5
Weather Normalized Site EUI (kBtu/ft²)	1465	12.5
Natural Gas Use (kBtu)	1442	12.3
Weather Normalized Site Electricity Intensity (kWh/ft²)	787	6.7
Weather Normalized Site Electricity (kWh)	786	6.7
Electricity Use - Grid Purchase (kBtu)	244	2.1
Site EUI (kBtu/ft²)	163	1.4
Source EUI (kBtu/ft²)	163	1.4
NYC Building Identification Number (BIN)	162	1.4
Street Number	124	1.1
Street Name	122	1.0
Water Required?	118	1.0
DOF Gross Floor Area	118	1.0
Borough	118	1.0
Direct GHG Emissions (Metric Tons CO2e)	83	0.7
Total GHG Emissions (Metric Tons CO2e)	74	0.6
Indirect GHG Emissions (Metric Tons CO2e)	65	0.6
Metered Areas (Energy)	57	0.5
DOF Benchmarking Submission Status	30	0.3

Оставшиеся пропуски необходимо заполнить, опираясь на одну из стратегий по заполнению пропусков, которые мы обсуждали на лекции.		
BBL - 10 digits	11	0.1
NYC Borough, Block and Lot (BBL) self-reported	11	0.1
Largest Property Use Type - Gross Floor Area (ft²)	2	0.0
✓ Заполнение пропусков Largest Property Use Type	2	0.0

Основная функция, которая используется для заполнения пропусков — это функция `.fillna()`

Она может принимать как одно значение (например -1):

```
data.fillna(-1)
```

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Postal Code	Street Number	Street Name	Borough	DOF Gross Floor Area	Primary Property Type – Self Selected	List of All Property Use Types at Property
0	1	13286	201/205	13286	201/205	1013160001	1013160001	1037549	201/205 East 42nd st.	10017	675	3 AVENUE	Manhattan	289356.0	Office	Office
1	2	28400	NYP Columbia (West Campus)	28400	NYP Columbia (West Campus)	1021380040	1-02138-0040	1084198; 1084387; 1084385; 1084386; 1084388; 10...	622 168th Street	10032	180	FT WASHINGTON AVENUE	Manhattan	3693539.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
2	3	4778226	MSCHoNY North	28400	NYP Columbia (West Campus)	1021380030	1-02138-0030	1063380	3975 Broadway	10032	3975	BROADWAY	Manhattan	152765.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
3	4	4778267	Herbert Irving Pavilion & Millstein Hospital	28400	NYP Columbia (West Campus)	1021390001	1-02139-0001	1087281; 1076746	161 Fort Washington Ave	10032	161	FT WASHINGTON AVENUE	Manhattan	891040.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
4	5	4778288	Neuro Institute	28400	NYP Columbia (West Campus)	1021390085	1-02139-0085	1063403	710 West 168th Street	10032	193	FT WASHINGTON AVENUE	Manhattan	211400.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11741	14988	4940453	Advanced: 24 Monroe Place	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3002380026	3-00238-0026\t	3001927	24 Monroe Place	11218	22	MONROE PLACE	Brooklyn	70645.0	Multifamily Housing	Multifamily Housing
11742	14989	4940416	Advanced: 150 Joralemon St / 124 Clinton St	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3002640017	3-00264-0017\t	3002539	150 Joralemon Street	11201	130	CLINTON STREET	Brooklyn	93500.0	Multifamily Housing	Multifamily Housing, Other
11743	14990	4628296	(9267) - 267 Sixth St	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3009870001	3-00987-0001	3413788; 3021326	267 6th Street	11215	-1	-1	-1	-1.0	Multifamily Housing	Multifamily Housing
11744	14991	4940464	Advanced: 27 Prospect Park West	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3010720040	3-01072-0040\t	3024968; 3824680	27 Prospect Park West	11215	27	PROSPECT PARK WEST	Brooklyn	57824.0	Multifamily Housing	Multifamily Housing
11745	14993	4952165	Tryad: 420 Clinton Ave	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3019600022	3-01960-0022\t	3055969	420 Clinton Ave\t	11238	419	VANDERBILT AVENUE	Brooklyn	60720.0	Multifamily Housing	Multifamily Housing

11746 rows x 49 columns

Так и `pd.Series`, у которого индексы являются названиями столбцов, а значения — это те значения, на которые мы хотим заменить пропуски.

Например, можно заменить пропуски максимальным значением в столбце, применив `.max()`

```
data.fillna(data.max())
```

▼ Вопрос

Почему возникла ошибка? И как её исправить?

```
data.fillna(data.max(numeric_only = True))
```

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Postal Code	Street Number	Street Name	Borough	DOF Gross Floor Area	Primary Property Type – Self Selected	List of All Property Use Types at Property
0	1	13286	201/205	13286	201/205	1013160001	1013160001	1037549	201/205 East 42nd st.	10017	675	3 AVENUE	Manhattan	289356.0	Office	Office
1	2	28400	NYP Columbia (West Campus)	28400	NYP Columbia (West Campus)	1021380040	1-02138-0040	1084198; 1084387; 1084385; 1084386; 1084388; 10...	622 168th Street	10032	180	FT WASHINGTON AVENUE	Manhattan	3693539.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
2	3	4778226	MSCHoNY North	28400	NYP Columbia (West Campus)	1021380030	1-02138-0030	1063380	3975 Broadway	10032	3975	BROADWAY	Manhattan	152765.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
3	4	4778267	Herbert Irving Pavilion & Millstein Hospital	28400	NYP Columbia (West Campus)	1021390001	1-02139-0001	1087281; 1076746	161 Fort Washington Ave	10032	161	FT WASHINGTON AVENUE	Manhattan	891040.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
4	5	4778288	Neuro Institute	28400	NYP Columbia (West Campus)	1021390085	1-02139-0085	1063403	710 West 168th Street	10032	193	FT WASHINGTON AVENUE	Manhattan	211400.0	Hospital (General Medical & Surgical)	Hospital (General Medical & Surgical)
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
11741	14988	4940453	Advanced: 24 Monroe Place	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3002380026	3-00238-0026\t	3001927	24 Monroe Place	11218	22	MONROE PLACE	Brooklyn	70645.0	Multifamily Housing	Multifamily Housing
11742	14989	4940416	Advanced: 150 Joralemon St / 124 Clinton St	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3002640017	3-00264-0017\t	3002539	150 Joralemon Street	11201	130	CLINTON STREET	Brooklyn	93500.0	Multifamily Housing	Multifamily Housing, Other
11743	14990	4628296	(9267) - 267 Sixth St	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3009870001	3-00987-0001	3413788; 3021326	267 6th Street	11215	NaN	NaN	NaN	13540113.0	Multifamily Housing	Multifamily Housing
11744	14991	4940464	Advanced: 27 Prospect Park West	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3010720040	3-01072-0040\t	3024968; 3824680	27 Prospect Park West	11215	27	PROSPECT PARK WEST	Brooklyn	57824.0	Multifamily Housing	Multifamily Housing
11745	14993	4952165	Tryad: 420 Clinton Ave	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3019600022	3-01960-0022\t	3055969	420 Clinton Ave\t	11238	419	VANDERBILT AVENUE	Brooklyn	60720.0	Multifamily Housing	Multifamily Housing

В нашем же случае давайте заменим пропуски в числовых признаках средним по колонке (`.mean()`), как и предыдущая функция, выдаст средние только для числовых столбцов)

```
data = data.fillna(data.mean(numeric_only = True))
```

В категориальных же признаках давайте будем использовать просто -1 в качестве символа пропуска.

```
# Поскольку этот код выполняется после предыдущей ячейки, то нам достаточно сделать fillna(-1), потому что пропуски в числовых признаках уже заполнены

data = data.fillna(-1)
```

Ну вот, всё получилось! Проверим, что пропусков действительно не осталось:

```
data.isna().any().any()

False
```

Разумеется, вы можете — да и это рекомендуется — в своих задачах придумать что-то более сложное, продвинутое и оригинальное, чем то, что мы рассмотрели выше. Сейчас же, напомним, наша с вами цель — лишь легонько коснуться EDA и изучить, какие в целом присутствуют этапы в первичном анализе данных и что в рамках этих этапов можно вообще делать!

✓ Univariate analysis

Теперь, когда утомительные, но совершенно необходимые этапы чистки данных и работы с пропусками закончены — можно углубляться в анализ.

Сделаем здесь небольшой шаг назад и кратко поговорим о EDA в целом.

Разведочный анализ данных (Exploratory Data Analysis, EDA) — это процесс, который можно продолжать до бесконечности: мы строим графики, ищем закономерности, детектируем аномалии, анализируем связи между признаками, а также в целом "погружаемся в данные".

Глобально, цель этого этапа в рамках анализа данных заключается в том, чтобы понять, что имеющиеся данные могут дать нам полезного в принципе. Обычно процесс начинается с обзора всего набора, затем переходит к его специфическим подмножествам и так далее. Любые находки могут быть по-своему интересны, также они могут дать нам ценные подсказки, например, по поводу относительной значимости различных признаков к целевой переменной или любые другие полезные инсайты.

✓ Анализ целевой переменной

Напомню, что глобальная цель — это предсказание значения целевого признака, рейтинга энергопотребления в нашем случае (кстати, давайте поэтому переименуем его в score в нашем наборе). Так что в рамках [Univariate analysis](#) целесообразно для начала понять, какое целевая переменная имеет распределение.

Посмотрим на него, построив гистограмму с matplotlib.

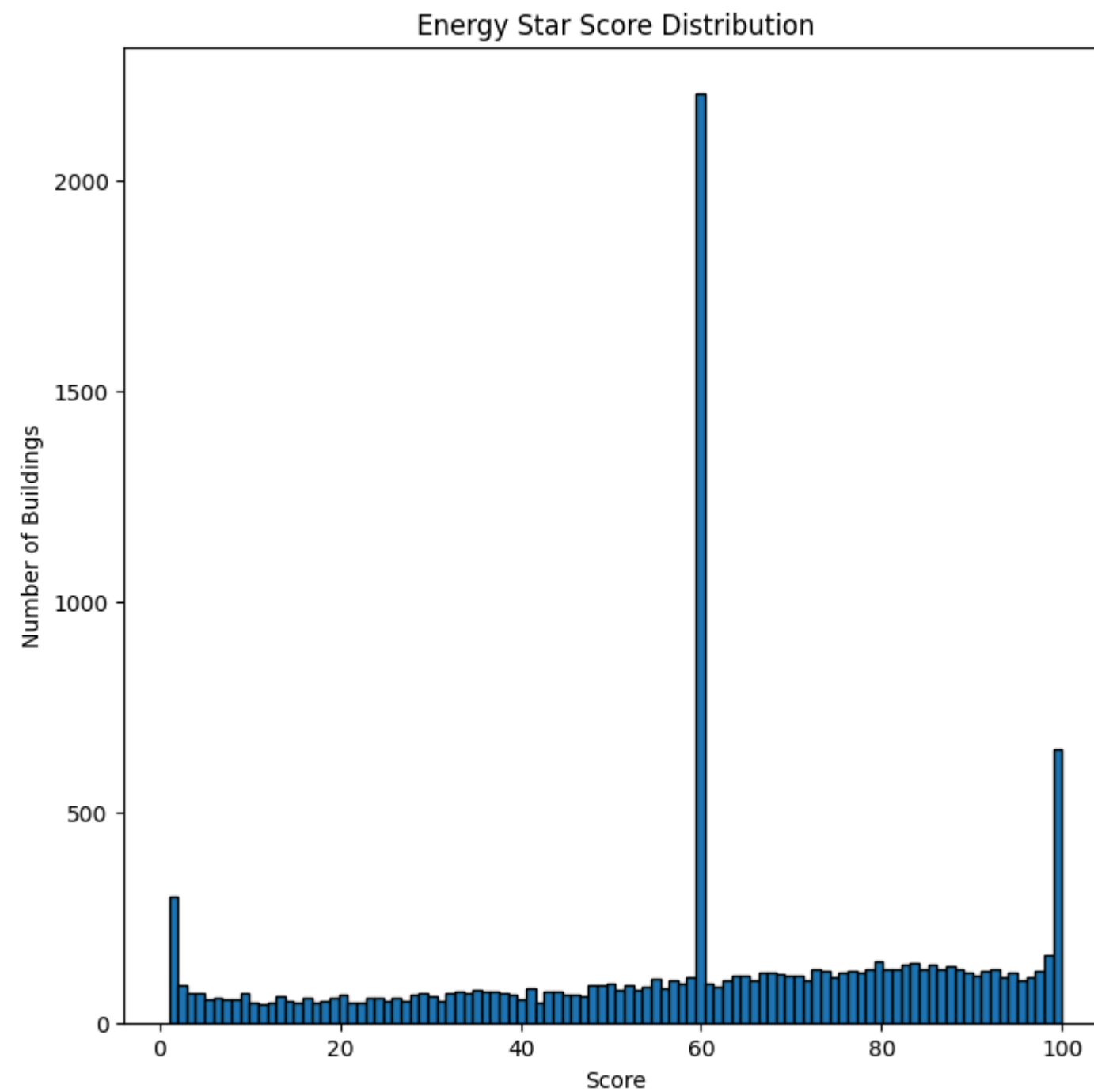
```
figsize(8, 8)

# Для удобства переименуем столбец 'ENERGY STAR Score' в 'score' при помощи метода rename
data = data.rename(columns = {'ENERGY STAR Score': 'score'})

# Построим гистограмму распределения столбца score, воспользовавшись функцией plt.hist, указав число бинов (например) 100
plt.hist(data['score'], bins = 100, edgecolor = 'k')

plt.xlabel('Score')
plt.ylabel('Number of Buildings')
plt.title('Energy Star Score Distribution')
#plt.style.use('fivethirtyeight')
```

```
Text(0.5, 1.0, 'Energy Star Score Distribution')
```



Вопрос

Почему распределение именно такое? Что вы можете сказать? Какие выводы напрашиваются?

Вывод

Пока выглядит довольно подозрительно!

Интересующий нас рейтинг представляет собой перцентиль, так что ожидаемо было бы увидеть равномерное распределение, где каждому значению соответствует примерно одинаковое количество зданий. Хотя в нашем случае на лицо диспропорция, больше всего зданий имеют максимальное значение рейтинга — 100, либо минимальное — 1 (высокий рейтинг это хороший показатель).

Обратимся к описанию признака и вспомним, что он основывается на “предоставляемых отчетах об энергопотреблении”. Это, возможно, кое-что объясняет. Просить владельцев зданий отчитаться об эффективности использования электроэнергии — это почти то же самое, что просить поставить студента оценку самому себе на экзамене. В результате мы получаем не самую объективную оценку эффективности использования электроэнергии в зданиях.

Если бы время не было ничем ограничено, стоило бы выяснить, почему большинство зданий имеют слишком высокие или слишком низкие значения рейтинга. Для этого нужно отфильтровать записи по этим зданиям и посмотреть, что у них общего. В нашу задачу не входит изобретение метода новой оценки эффективности энергопотребления, так что лучше пока сфокусироваться на предсказании рейтинга с тем, что есть.

data[data['score'] <= 1].head(2)

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Postal Code	Street Number	Street Name	Borough	DOF Gross Floor Area	Primary Property Type – Self Selected	List of All Property Use Types at Property	Large Proper Use Ty
13	16	5871253	Cadman - 10 Clinton St	2637863	Cadman Towers	3002380035	3002380035	3319420	101 Clark St	11201	10	CLINTON STREET	Brooklyn	212656.0	Multifamily Housing	Multifamily Housing, Parking, Retail Store	Multifam Housi
62	65	2658585	CEPSR Schapiro	3612678	Columbia University (morningside)	1019730001	1019730001	1084476	530 West 120th Street	10027	530	WEST 120 STREET	Manhattan	1368870.0	Office	Office	Off

Анализ нецелевых переменных

Теперь давайте проведем исследование для других переменных (нецелевых).

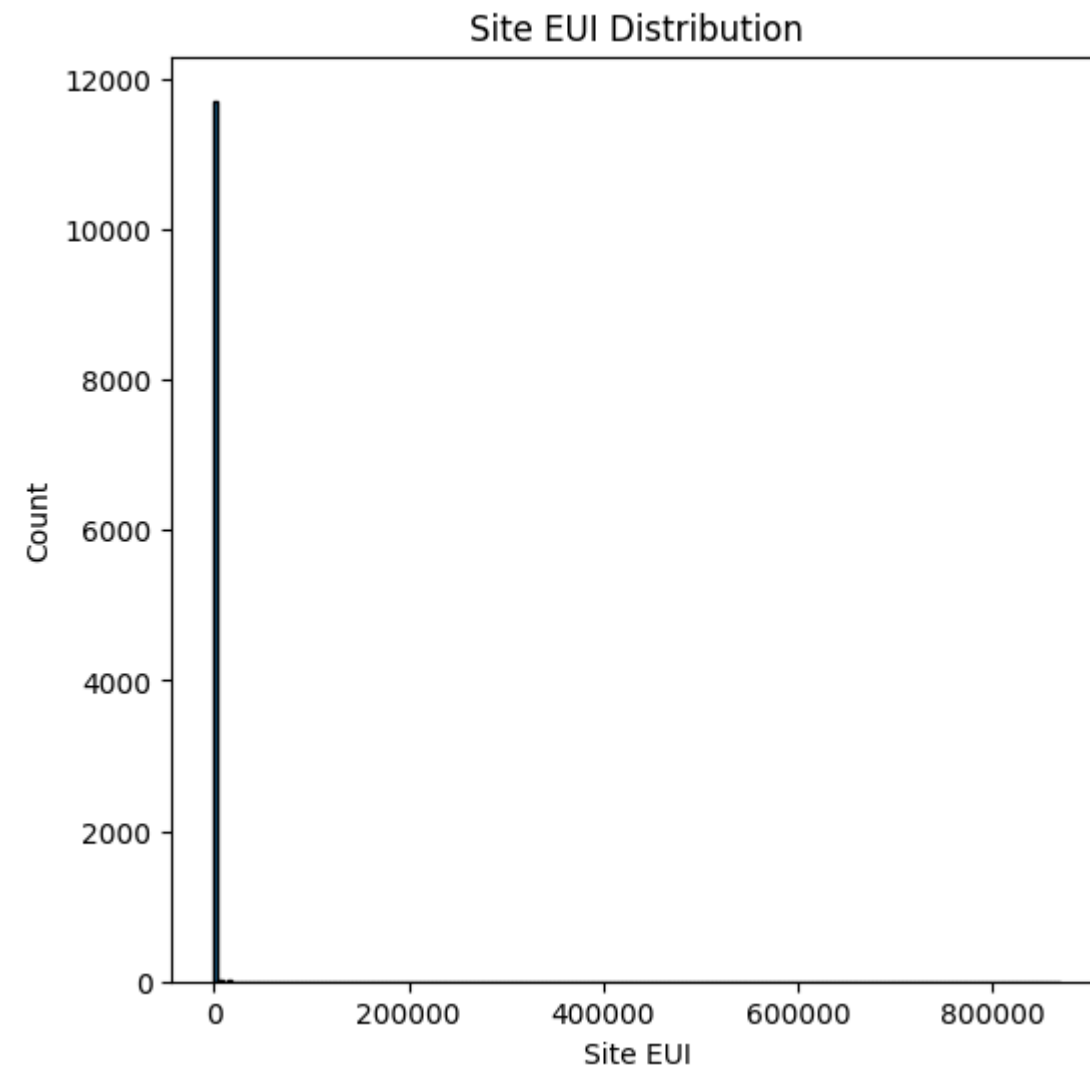
Построим гистограмму распределения поля Site EU1, воспользовавшись функцией `plt.hist`, указав число бинов (например) 200

```
figsize(6, 6)

plt.hist(data['Site EUI (kBtu/ft²)'], bins = 200, edgecolor = 'black')

plt.xlabel('Site EUI')
plt.ylabel('Count')
plt.title('Site EUI Distribution')
```

```
Text(0.5, 1.0, 'Site EUI Distribution')
```



Что можно увидеть на этой гистограмме? Что вообще происходит на графике?

Здесь, на этом графике мы видим явные выбросы! Из-за этого бедную гистограмму так сильно сплющило...

Посмотрим на статистики этой же колонки, воспользовавшись методом `describe()`

```
data['Site EUI (kBtu/ft²)'].describe()
```

Site EUI (kBtu/ft²)	
count	11746.000000
mean	280.071484
std	8547.243929
min	0.000000
25%	62.100000
50%	79.000000
75%	98.800000
max	869265.000000
dtype: float64	

Только посмотрите на то, какой разброс между третьим квартилем и максимумом...

Посмотрим на топ 10 самых больших значений в этой колонке:

data['Site EUI (kBtu/ft²)'].dropna().sort_values().tail(10)	
Site EUI (kBtu/ft²)	
3173	51328.8
3170	51831.2
3383	78360.1
8269	84969.6
3263	95560.2
8268	103562.7
8174	112173.6
3898	126307.4
7	143974.4
8068	869265.0
dtype: float64	

Давайте посмотрим на то, сколько вообще существует объектов со значением метрики больше 1000:

data[data['Site EUI (kBtu/ft²)'] > 1000].shape[0]
47

Их всего 47!

Повнимательнее посмотрим на эти выбросы:

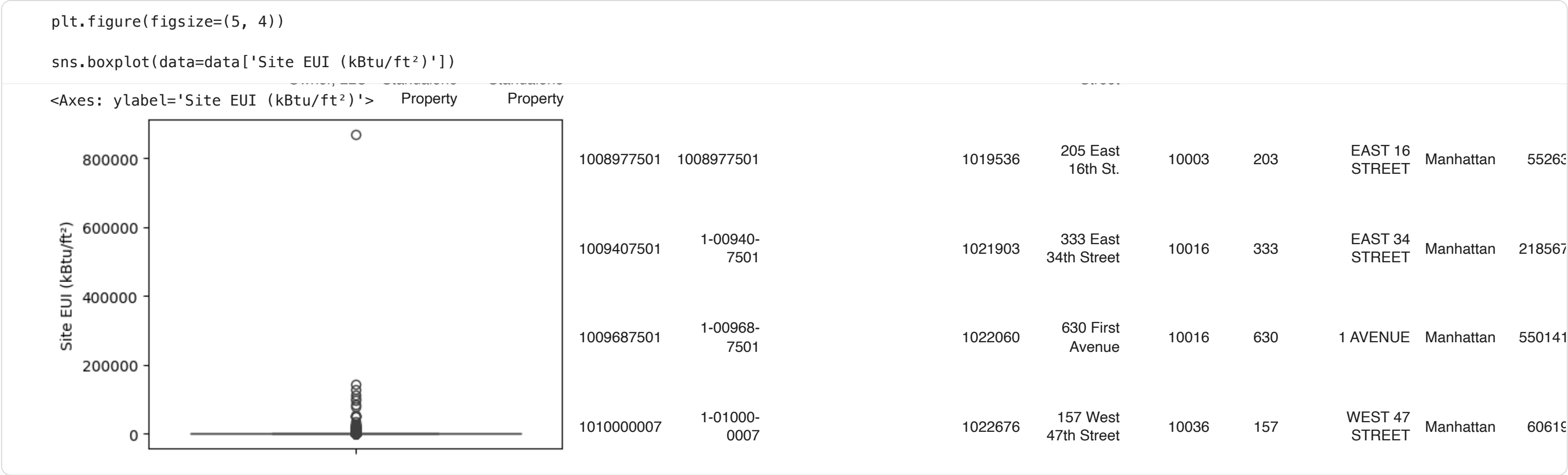
```
data.loc[data['Site EUI (kBtu/ft²)'] > 1000, :]
```

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Postal Code	Street Number	Street Name	Borough	Dist. Gross Floor Area	
	7	10	2610789	North Shore Towers	2610789	North Shore Towers	4084890001	4084890001	4456886;4456885;4453535;4456888	270-10 Grand Central Parkway	11005	269	GRAND CENTRAL PKWY	Queens	3750565
	8	11	2611745	Towers Golf Course and Irrigation Wells	2610789	North Shore Towers	4084890001	4084890001	4456888	272-86 Grand Central Parkway	11005	269	GRAND CENTRAL PKWY	Queens	3750565
	62	65	2658585	CEPSR Schapiro	3612678	Columbia University (morningside)	1019730001	1019730001	1084476	530 West 120th Street	10027	530	WEST 120 STREET	Manhattan	1368870
	123	126	2826188	Warren Weaver Hall	3616399	New York University: Washington Square	1005350036	1005350036	1008627	251 Mercer Street	10012	17	WEST 3 STREET	Manhattan	1191700
	541	585	2714770	zzzWassner 86 Trinity Place/ 113 Greenwich	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1000510013	1-00051-0013	1001033	86 Trinity Place	10006	86	TRINITY PLACE	Manhattan	1817250
	546	592	4039500	W New York - Downtown	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1000537502	1-00053-7502	1088442	123 Washington Street	10006	123	WASHINGTON STREET	Manhattan	3964770
	564	613	2796479	75 Maiden	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1000680011	1-00068-0011	1001112	75 Maiden Lane	10038	75	MAIDEN LANE	Manhattan	1550260
	847	1033	4412906	The Door 121 Ave of the Americas	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1004777501	1-00477-7501	1083493	121 Ave of the Americas	10013	121	AVENUE OF THE AMER	Manhattan	8086200
	1256	1520	2611862	111 8th Avenue	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1007390001	1007390001	1013043	111 8th Avenue	10011	111	8 AVENUE	Manhattan	2161994
	1276	1546	3998801	Technical Career Institute	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1007540051	1-00754-0051	1013537	320 w 31 street	10001	320	WEST 31 STREET	Manhattan	1160000

Есть ли какие-то закономерности в данных? Можно ли сделать какие-то выводы касательно этих зданий?

Воспользуемся boxplot-ом из библиотеки seaborn для того, чтобы убедиться, что эти объекты действительно похожи на выбросы

1589	1903	3897375	HAIE01356 Haler America Building 1356	Not Applicable: Standalone	Not Applicable: Standalone	1008120029	1-00812-0029	1015241	1356 Broadway	10018	1352	BROADWAY	Manhattan	76804
------	------	---------	---	----------------------------------	----------------------------------	------------	--------------	---------	------------------	-------	------	----------	-----------	-------

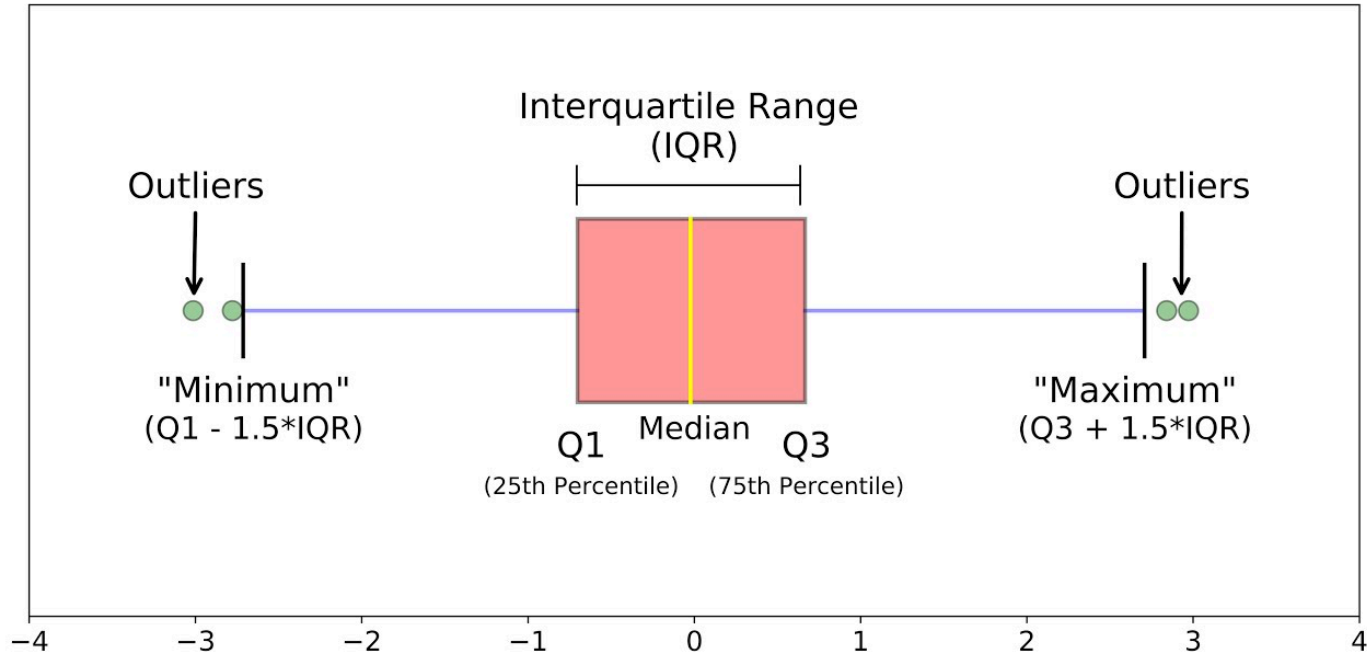


Что же опять пошло не так?

2367	2771	3280228	The Westin NY at Times Square	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1010140058	1-01014-0058	1087141	270 West 43rd Street	10036	270	WEST 43 STREET	Manhattan	690000
------	------	---------	-------------------------------	--	--	------------	--------------	---------	----------------------	-------	-----	----------------	-----------	--------

Reminder: BoxPlot

3100	3589	4536727	JSRE 546 Fifth Ave	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1012610034	1-01261-0034	1082895	546 Fifth Avenue	10036	546	5 AVENUE	Manhattan	180524
3139	3628	5851622	The Jewel	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1012677502	1-01267-7502	-1	11 W 51 Street	10019	11	WEST 51 STREET	Manhattan	64727
3163	3652	4390423	10 W 57th Street	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1012720047	1-01272-0047	1081138	10 W 57th Street	10019	10	WEST 57 STREET	Manhattan	81174
3170	3660	4390422	9 W 57th Street	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1012730022	1-01273-0022	1035071	9 W 57th Street	10019	9	WEST 57 STREET	Manhattan	1383760
3173	3663	4390425	4 W 58th Street	Not Applicable: Standalone Property	Not Applicable: Standalone Property	1012730044	1-01273-0044	1035074	4 W 58th Street	10019	4	WEST 58 STREET	Manhattan	61000
3263	3760	4043349	Rolex Realty Company Inc	Not Applicable: Standalone	Not Applicable: Standalone	1012880069	1012880069	1035721	665 Fifth Avenue	10022	665	5 AVENUE	Manhattan	122596



Box plot — график, использующийся в оппительной статистике, компактно изображающий одномерное распределение вероятностей.

Такой вид диаграммы в удобной форме показывает медиану (или, если нужно, среднее), нижний и верхний квартили, минимальное и максимальное значение выборки и выбросы.

Несколько таких ящиков можно нарисовать бок о бок, чтобы можно было сравнить одно распределение с другим; их можно располагать как горизонтально, так и вертикально. Расстояния между различными частями ящика позволяют определить степень разброса (дисперсии) и асимметрии данных и выявить выбросы.

Работа с выбросами

Для того, чтобы качественно осуществлять Univariate analysis, конечно же, нужно научиться избавляться от выбросов после их выявления.

Вообще, выбросы ([outliers](#)) могут быть связаны с разными причинами: опечатки, ошибки в единицах измерения или даже корректные, но чересчур экстремальные значения.

Удалять выбросы можно разными способами, но один из самых популярных работает по следующей схеме:

- Ниже первой квартили - $3 \cdot iqr$ (межквартильное расстояние)
- Выше третьей квартили + $3 \cdot iqr$ (межквартильное расстояние)

При этом 3 не является какой-либо мировой константой и вполне может варьироваться от задачи к задаче.

```
# Рассчитаем первый и третий квартили поля Site EUI или возьмем значения из вывода метода describe
first_quartile = data['Site EUI (kBtu/ft²)'].describe()['25%']
third_quartile = data['Site EUI (kBtu/ft²)'].describe()['75%']
```

[illegible]

	Site EUI (kBtu/ft²)				Standalone Property	Standalone Property				avenue			AVENUE		
0 81741	10187	4022803	False	7-29 HEGEMAN LLC	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3036210001	3-03621-0001	3082700	7-29 hege man avenue	11212	7	HEGEMAN AVENUE	Brooklyn	158620
2 82683 82684	10385	4022808	False	C GROSS	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3047180036	3-04718-0036	3103619	505 ROCKAWAY PARKWAY	11212	505	ROCKAWAY PARKWAY	Brooklyn	54000
...			...												
11741			True	GROSS C	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3047570001	3-04757-0001	3105176	660 EAST 98TH STREET	11236	638	EAST 98 STREET	Brooklyn	121472
11742			True												
11743 8924 11744	11177	3521991	True	2000 Realty Associates LLC	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3063340006	3-06334-0006	3165376	2000 84th Street	11204	2000	84 STREET	Brooklyn	90000
11745			True												
11746 rows x 1 columns 9057	11331	4616061		Terrace Gardens - E915	Not Applicable: Standalone Property	Not Applicable: Standalone Property	3067010064	3067010064	3179403	915 EAST 17TH STREET	11230	915	EAST 17 STREET	Brooklyn	136800
dtype: bool															

Применение	10175	12775	193023	193023	Not Applicable: Standalone Property	Not Applicable: Standalone Property	401350043	4015560043	4038913	42-25 Hampton St	11373	42-11	HAMPTON STREET	Queens	73100
------------	-------	-------	--------	--------	-------------------------------------	-------------------------------------	-----------	------------	---------	------------------	-------	-------	----------------	--------	-------

10710	10101	5705170	Golden Gate	Applicable:	Applicable:	1000500000	1000500000	1000110	Golden Gate	11115	0000	KEW GARDENS	000000	00701
-------	-------	---------	-------------	-------------	-------------	------------	------------	---------	-------------	-------	------	-------------	--------	-------

(11325, 49) 11186, 14064	2815494	Glen Oaks Village Section B	Applicable: Standalone	Applicable: Standalone	4084900002	4084900002	4456754	245-24 76th Ave	11426	245-24	76 AVENUE	Queens	123120
-----------------------------	---------	-----------------------------------	---------------------------	---------------------------	------------	------------	---------	--------------------	-------	--------	-----------	--------	--------

11189	14070	2815534	Village Section E	Standalone Property	Standalone Property	4085150002	4085150002	4454005	Street	110041199	74-02	260 STREET	Queens	137476
11504	14570	5764630	144-01 157th Street	Not Applicable: Standalone Property	Not Applicable: Standalone Property	4150120006	4-15012-0006	4296646	144-01 157th Street	11434	144-01	157 STREET	Queens	57811

	Order	Property Id	Property Name	Parent Property Id	Parent Property Name	BBL – 10 digits	NYC Borough, Block and Lot (BBL) self-reported	NYC Building Identification Number (BIN)	Address 1 (self-reported)	Postal Code	Street Number	Street
12	15	2637863	Cadman Towers	2637863	Cadman Towers	3002320001;3002380035	3002320001;3002380035	3001727;3319420	101 Clark St	11201	-1	
17	20	2777309	Barnard College	2777309	Barnard College	1019890001	1-01989-0001	1082351;1079204;1082351;1082347;1079205;1082350	3009 Broadway	10029	-1	
18	21	2780056	720-730 Ft. Washington Owners Corp.	2780056	720-730 Ft. Washington Owners Corp.	1021800614;1021800628	1021800614;1021800628	1064475;1064476;1803806	720-730 Ft. Washington Ave.	10040	-1	
19	22	4988574	- 720 Ft. Washington Ave. bldg	2780056	720-730 Ft. Washington Owners Corp.	1021800614	1021800614	1064475	720-730 Ft. Washington Ave.	10040	720	WASHING AVE
20	23	4988569	- 730 Ft. Washington Ave bldg	2780056	720-730 Ft. Washington Owners Corp.	1021800628	1021800628	1064476;1803806	720-730 Ft. Washington Ave.	10040	730	WASHING AVE

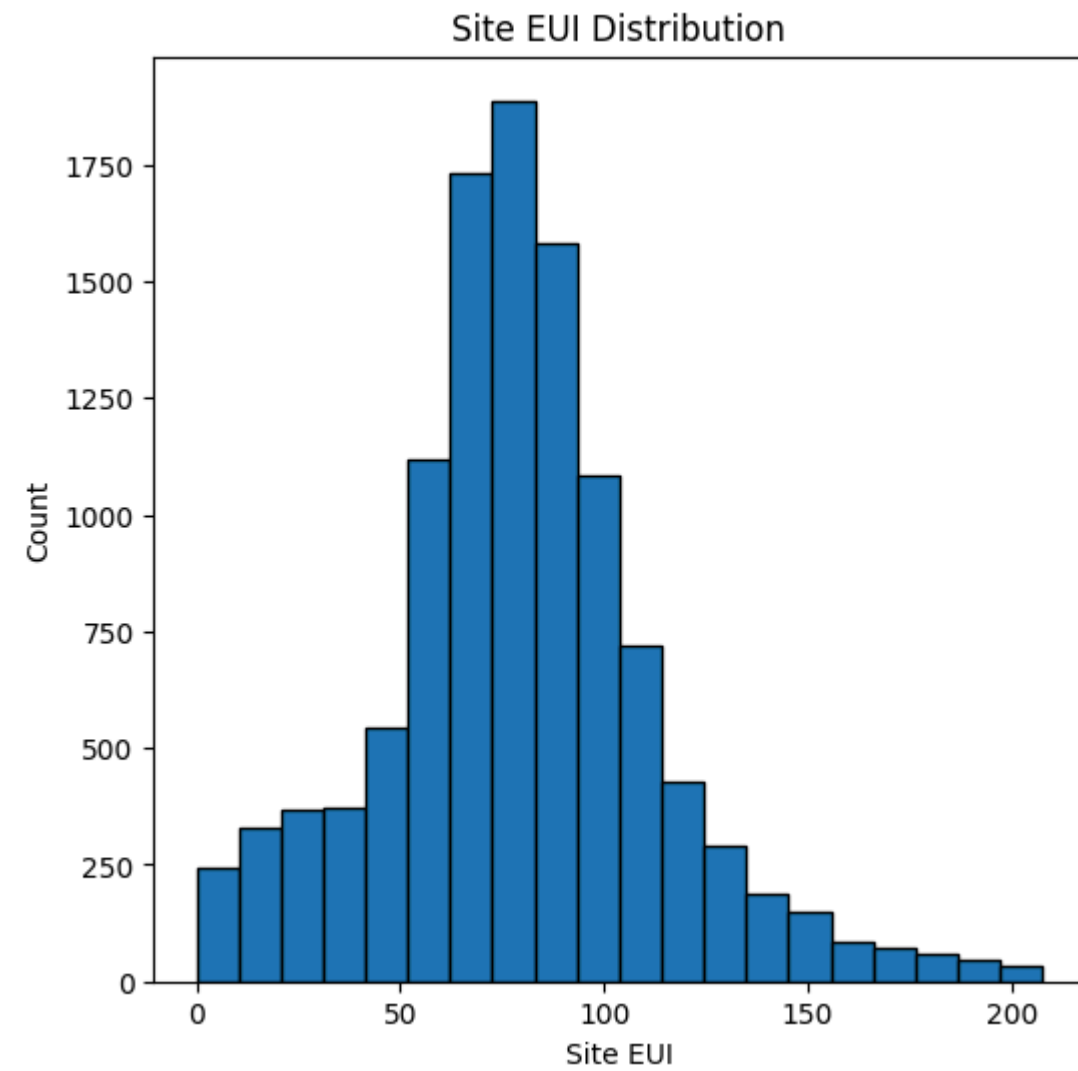
Вроде выглядит хорошо! Давайте построим теперь еще раз гистограмму распределения поля Site EUI, воспользовавшись функцией `plt.hist`, указав число бинов (например) 20. Проверим, насколько наши меры по удалению выбросов в действительности помогли!

```
figsize(6, 6)

plt.hist(data['Site EUI (kBtu/ft²)'], bins = 20, edgecolor = 'black')

plt.xlabel('Site EUI')
plt.ylabel('Count')
plt.title('Site EUI Distribution')
```

```
Text(0.5, 1.0, 'Site EUI Distribution')
```

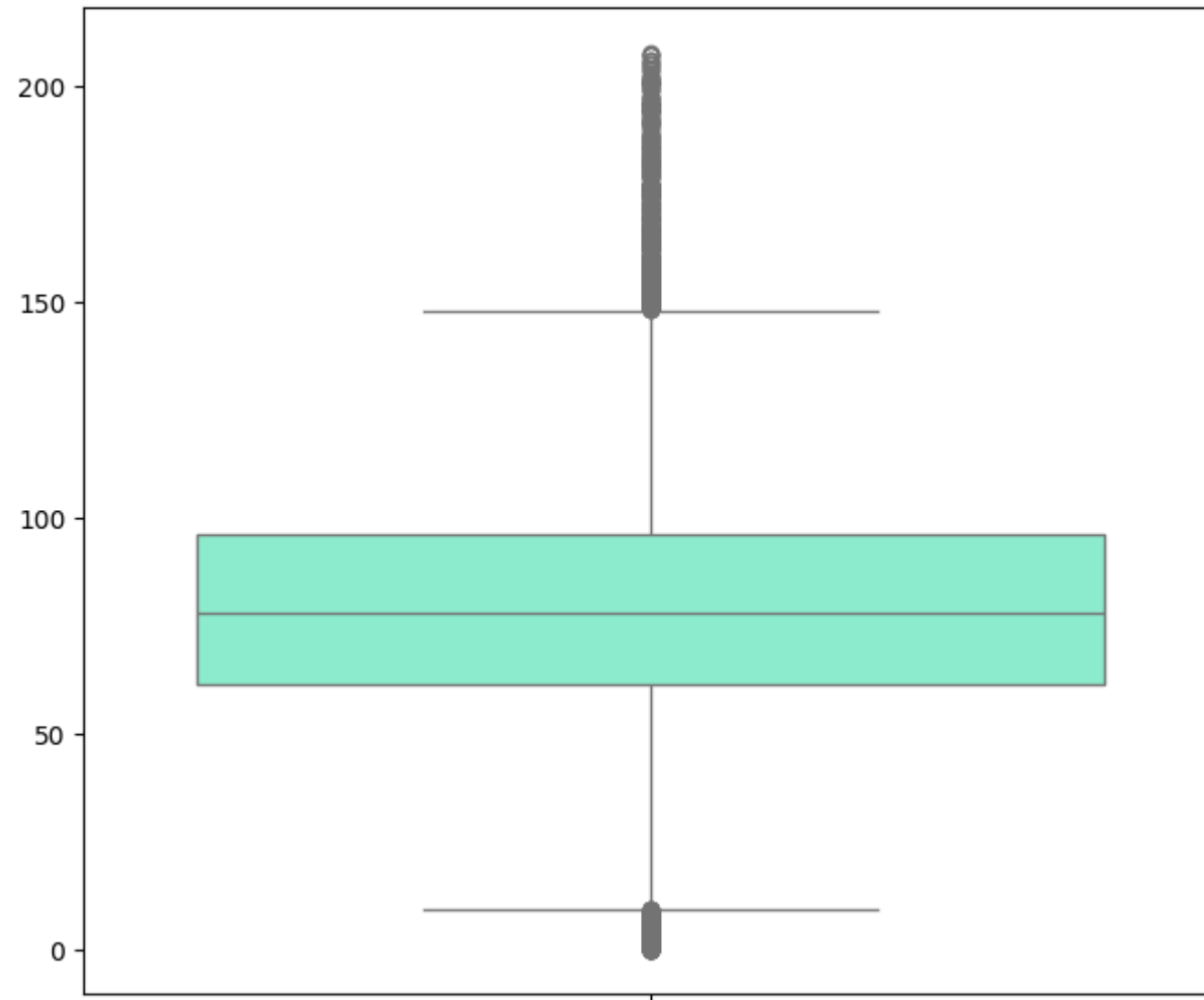


Потрясающе! Это даже очень напоминает нормальное распределение, что совсем здорово для наших данных!

Воспользуемся теперь еще раз `boxplot`-ом из библиотеки `seaborn` для того, чтобы убедиться, что выбросы пропали и что мы всё сделали правильно!

```
plt.figure(figsize=(8, 7))  
  
sns.boxplot(data=data['Site EUI (kBtu/ft²)'].values, color = 'aquamarine')
```

<Axes: >



Действительно потрясающая работа! Мы молодцы! :)

Вопрос

Почему же, если работа такая хорошая, box-plot до сих пор детектирует какие-то выбросы на графике?

✓ Другие способы борьбы с выбросами

Помимо того способа, который мы использовали, есть следующие способы борьбы с выбросами:

- **Dropping the outlier rows with standard deviation**

```
factor = 3
upper_lim = data['column'].mean() + data['column'].std() * factor
lower_lim = data['column'].mean() - data['column'].std() * factor
data = data[(data['column'] < upper_lim) & (data['column'] > lower_lim)]
```

- **Dropping the outlier rows with Percentiles**

```
upper_lim = data['column'].quantile(.95)
lower_lim = data['column'].quantile(.05)
data = data[(data['column'] < upper_lim) & (data['column'] > lower_lim)]
```

Примеры других способов:

- Винсоризация — это серия трансформаций, направленных на ограничение влияния выбросов. 90%-ая винсоризация означает, что мы берём значения меньше 5% перцентиля и выше 95% перцентиля и приравниваем их к значениям на 5-м и 95-м перцентилях соответственно.
- Триминг — отличается от винсоризации тем, что мы не ограничиваем крайние значения каким-либо числом, а просто удаляем их.

Больше методов вы можете найти в [scipy.stats.mstats](https://docs.scipy.org/doc/scipy/reference/stats.html)

Ну а мы — возвращаемся назад к анализу :)

✓ Multivariate analysis

Значительную часть работы на этапе EDA занимает поиск взаимосвязей между различными признаками. Очевидно, что признаки и значения признаков, оказывающие основное влияние на целевой, интересуют нас сильнее, чем прочие — по ним лучше всего и предсказывать значение целевого.

✓ Влияние категориальной переменной на целевую

Одним из способов оценить влияние значений категориальных признаков (число значений такого признака подразумевается конечным) на целевой признак является построение графика density plot, например, используя модуль seaborn. Density plot можно представить себе как сглаженную гистограмму или же функцию плотности для некоторого распределения.

При применении этой визуализации в связке целевой переменной и нецелевой категориальной, мы получаем распределение целевого признака для одной конкретной категории.

Код ниже строит density plot для рейтинга энергопотребления в зависимости от типа здания. Разными цветами показаны рейтинги для различных типов (при этом к рассмотрению берутся только типы зданий с как минимум сотней записей в нашем наборе данных):

```
# Создадим список types с названиями категорий зданий с более, чем 100 измерениями
types = data.dropna(subset=['score'])
types = types['Largest Property Use Type'].value_counts()
types = list(types[types.values > 100].index)
types.remove('Other')
```

```

figsize(10, 6)

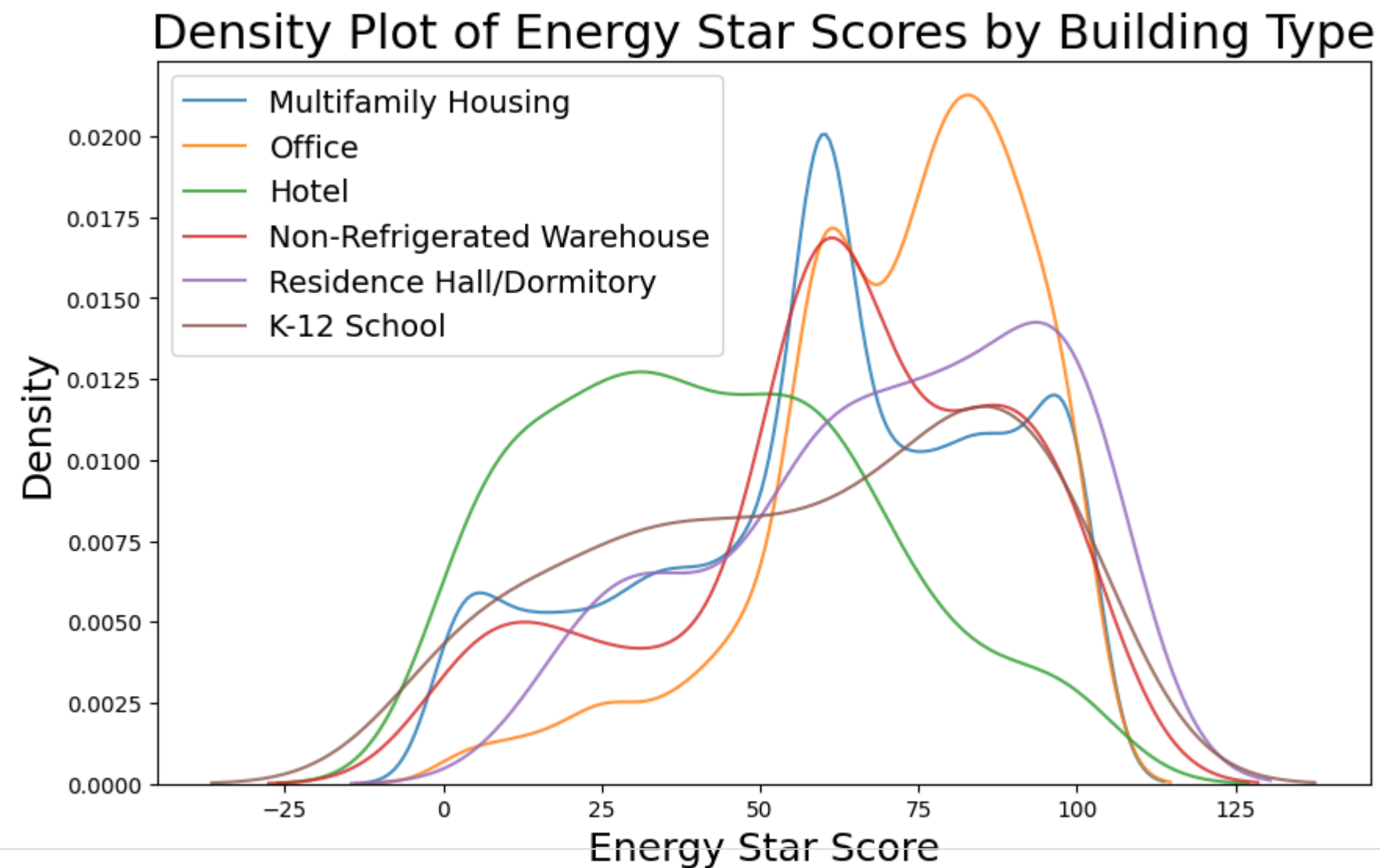
# Постройте на одном графике распределения оценок для каждого типа зданий из списка types
for b_type in types:
    # Выберите данные одного из типов зданий из списка types
    subset = data[data['Largest Property Use Type'] == b_type]

    # Воспользуйтесь функцией kdeplot библиотеки seaborn для отрисовки распределения
    sns.kdeplot(subset['score'], label = b_type, fill = False, alpha = 0.8)

plt.xlabel('Energy Star Score', size = 18)
plt.ylabel('Density', size = 18)
plt.title('Density Plot of Energy Star Scores by Building Type', size = 22)
plt.legend(fontsize=14)

```

<matplotlib.legend.Legend at 0x7cdba6c3b890>



Какие результаты вы наблюдаете? Какие выводы можно сделать из графика?

Видно, что тип здания оказывает существенное влияние на рейтинг энергопотребления. Здания, используемые как офисы, чаще имеют хороший рейтинг, а отели наоборот. Получается, такой признак, как тип здания, для нас важен. Так как это признак категориальный, нам ещё предстоит выполнить с ним так называемый «one-hot encoding». Но это позже.

Давайте теперь повторим аналогичные действия для различных районов:

```
# Создадим список boroughs с названиями районов с более, чем 100 измерениями
boroughs = data.dropna(subset=['score'])
boroughs = boroughs['Borough'].value_counts()
boroughs = list(boroughs[boroughs.values > 100].index)
```

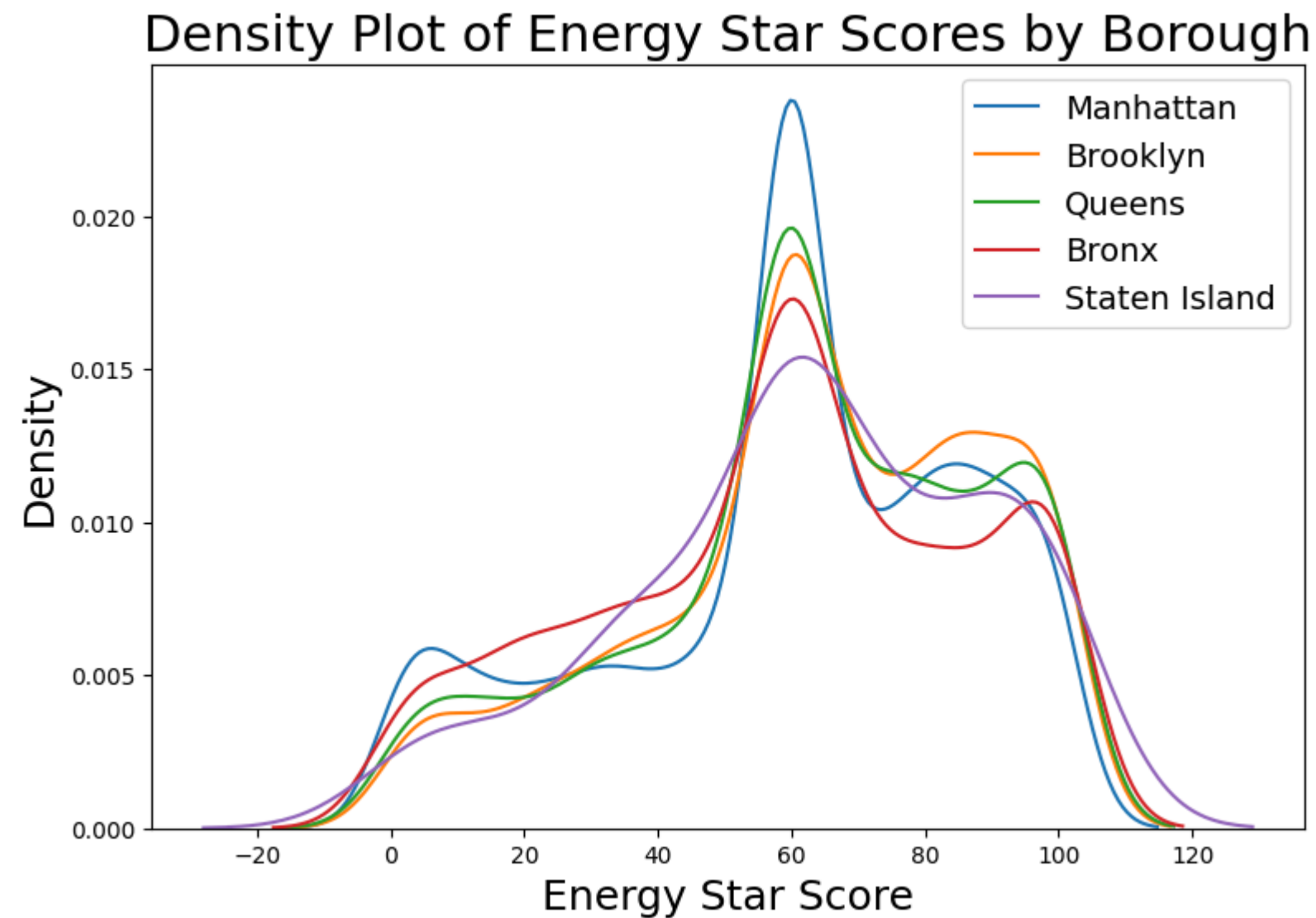
```
figsize(9, 6)
```

```
# Постройте на одном графике распределения оценок для каждого района из списка boroughs
for borough in boroughs:
    # Выберите данные одного из районов из списка boroughs
    subset = data[data['Borough'] == borough]
```

```
    # Воспользуйтесь функцией kdeplot библиотеки seaborn для отрисовки распределения
    sns.kdeplot(subset['score'], label = borough)
```

```
plt.xlabel('Energy Star Score', size = 18)
plt.ylabel('Density', size = 18)
plt.title('Density Plot of Energy Star Scores by Borough', size = 22)
plt.legend(fontsize=14)
```

<matplotlib.legend.Legend at 0x7cdba6b8d7d0>



А какие выводы можно сделать здесь?

Похоже, что район оказывает уже не такое большое влияние. Тем не менее, пожалуй, стоит включить этот признак в модель, так как определенная разница между районами все же есть.

Вопрос

А теперь вопрос, так сказать, со звездочкой и одновременно с этим очень-очень важный. Не видите ли вы каких-то проблем с тем, как мы построили последние два графика?

✓ Пояснение

Мы заполнили пропуски средним во **всех** числовых столбцах, **в том числе** в нашей целевой переменной. Из-за этого распределения ответов могли существенно поменяться... Нам с самого начала следовало строить распределение только по заполненным исходным строкам целевой переменной, даже если в других столбцах мы проделывали для таких строк заполнение средним!

Повторим быстренько пайплайн, скорректировав наши действия в необходимом месте.

```
data = pd.read_csv('https://raw.githubusercontent.com/ddvika/DS_2021/main/lecture_2/Energy_consumption_NY.csv')

data = data.replace({'Not Available': np.nan})

for col in list(data.columns):
    if any([n_col in col for n_col in part_name_numeric_cols]):
        data[col] = data[col].astype(float)

data = data.drop(columns = list(missing_columns))

data = data[condition]

###
score_col = data['ENERGY STAR Score']
data.drop('ENERGY STAR Score', axis = 1, inplace = True)

data = data.fillna(data.mean(numeric_only = True))
data = data.fillna(-1)

data['score'] = score_col
###
```

Повторим построения двух последних графиков теперь! И сравним!

```
types = data['Largest Property Use Type'].value_counts()
types = list(types[types.values > 100].index)
types.remove('Other')
```

```
figsize(10, 6)
```

```
# Постройте на одном графике распределения оценок для каждого типа зданий из списка types
```

```
for b_type in types:
```

```
    # Выберите данные одного из типов зданий из списка types
```

```
    subset = data[data['Largest Property Use Type'] == b_type]
```

```
    # Воспользуйтесь функцией kdeplot библиотеки seaborn для отрисовки распределения
```

```
    sns.kdeplot(subset['score'], label = b_type, fill = False, alpha = 0.8)
```

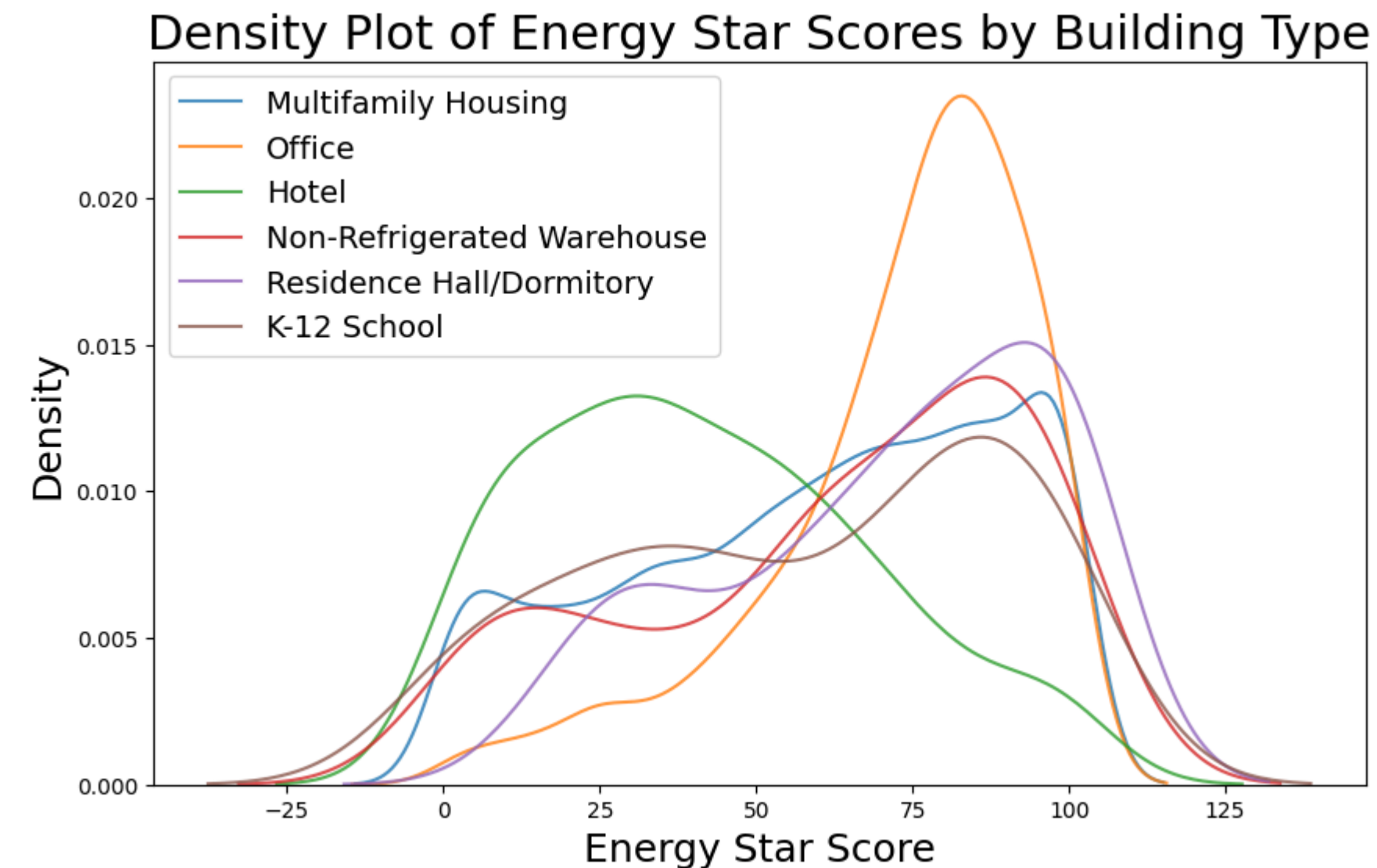
```
plt.xlabel('Energy Star Score', size = 18)
```

```
plt.ylabel('Density', size = 18)
```

```
plt.title('Density Plot of Energy Star Scores by Building Type', size = 22)
```

```
plt.legend(fontsize=14)
```

```
<matplotlib.legend.Legend at 0x7cdba23e7210>
```



```
boroughs = data['Borough'].value_counts()
boroughs = list(boroughs[boroughs.values > 100].index)
```

```
figsize(9, 6)
```

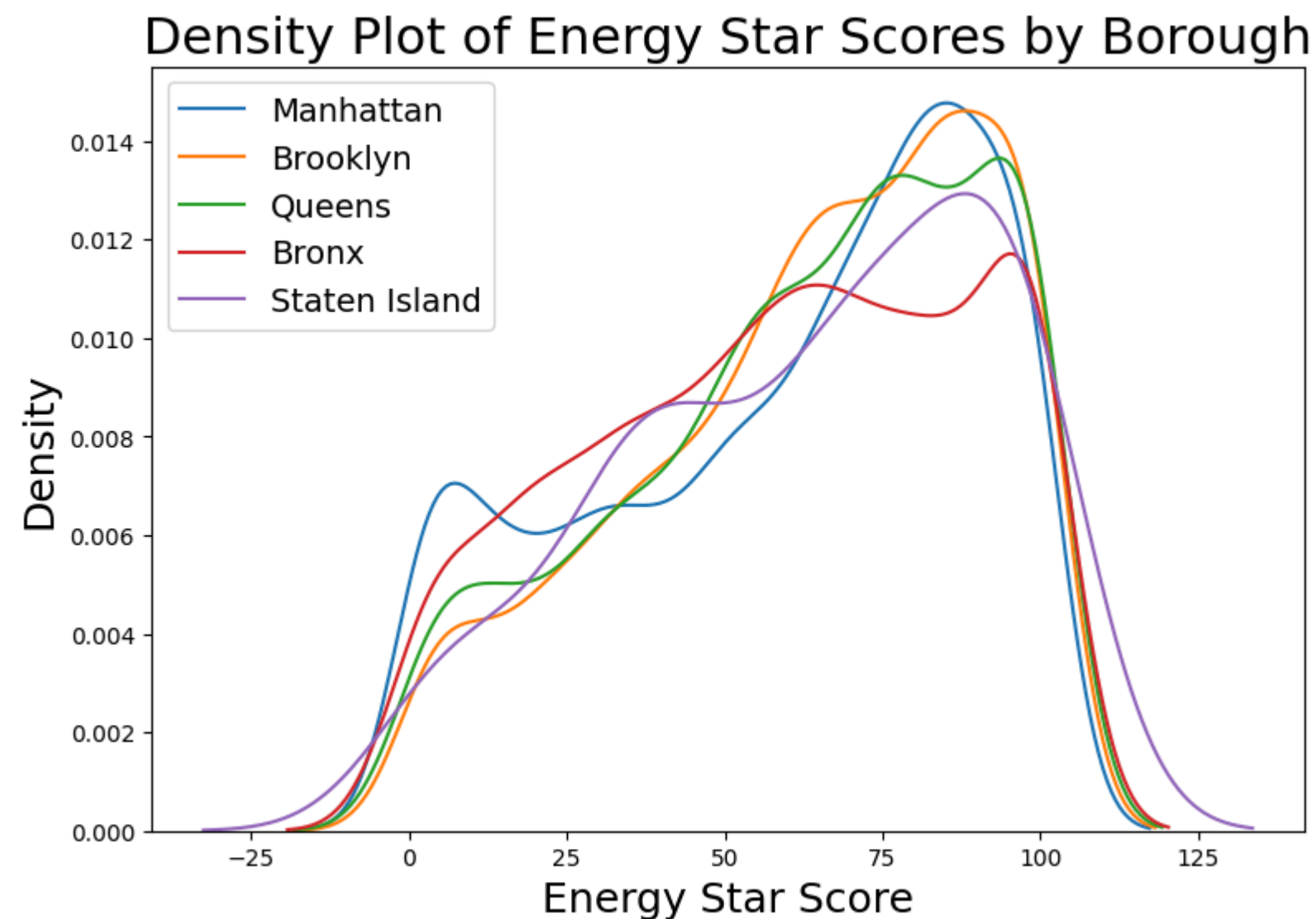
```
# Постройте на одном графике распределения оценок для каждого района из списка boroughs
for borough in boroughs:
```

```
    # Выберите данные одного из районов из списка boroughs
    subset = data[data['Borough'] == borough]
```

```
    # Воспользуйтесь функцией kdeplot библиотеки seaborn для отрисовки распределения
    sns.kdeplot(subset['score'], label = borough)
```

```
plt.xlabel('Energy Star Score', size = 18)
plt.ylabel('Density', size = 18)
plt.title('Density Plot of Energy Star Scores by Borough', size = 22)
plt.legend(fontsize=14)
```

<matplotlib.legend.Legend at 0x7cdba1dd4950>



Ну что — разница как будто бы есть! :)

На таких графиках можно делать уже гораздо более значимые и осмысленные выводы!